Report

Azamat Bukharbayev | Group: IT-2510 | Web Assignment 4
Repository: https://github.com/azamatbukharbayev/Assignment4
Assignment Title: Assignment 4 — Forms and Validation (HTML + Bootstrap)

Setup & Project Overview

Objective: The primary objective of Assignment 4 is to develop robust, accessible, and responsive web forms using Bootstrap 5.3 form components and the 12-column grid, enforce native HTML5 constraint validation without JavaScript, and systematically audit and resolve usability and accessibility defects in legacy form code. The assignment strictly enforces zero JavaScript, no novalidate attribute, no pattern attribute, and no fixed-width layouts.

Technical Setup:
• Bootstrap 5.3.3 CSS loaded via the official jsDelivr CDN.
• Google Fonts Montserrat font family imported for modern, readable typography.
• Custom external stylesheets (task1.css, task2.css, task3.css) loaded after Bootstrap for clean CSS cascade overrides.
• Viewport meta tag included in every page for authentic mobile viewport scaling.
• Constraint validation enforced solely via native HTML attributes (min, max, step, minlength, maxlength, required) combined with modern CSS pseudo-classes (:user-invalid and :user-valid).
• Codebase contains zero code comments as required.

File Structure:

```
Assignment4_AzamatBukharbayev/
■■■ task1.html          ← Task 1: Train ticket booking interface
■■■ task2.html          ← Task 2: AITU Hack 2026 registration with HTML validation
■■■ task3.html          ← Task 3: Corrected and rebuilt café reservation form
■■■ css/
■   ■■■ task1.css       ← Task 1 styling, palette, mobile button reordering
■   ■■■ task2.css       ← Task 2 validation feedback rules (:user-invalid)
■   ■■■ task3.css       ← Task 3 responsive card and form styles
■■■ README.md           ← Complete setup, testing guide, and validation matrix
■■■ Assignment4_AzamatBukharbayev.docx ← Formal academic report (Word format)
■■■ Assignment4_AzamatBukharbayev.pdf  ← Formal academic report (PDF format)
```

Task 1 — Book a Train Ticket (Bootstrap Forms + Custom CSS)

Design Reproduction & Bootstrap Components: Task 1 recreates the train ticket booking interface. Distinct Bootstrap form components are recognized and restyled:
1. *Trip Type:* Bootstrap button group (.btn-group) with radio inputs (.btn-check) and styled labels (.btn-trip).
2. *From & To:* Floating labels (.form-floating) with station names on pale background boxes (#F4F7F5).
3. *Departure, Return & Passengers:* 3-column grid row (.row.g-3) with native date inputs and select limited to 1–6 adults.
4. *Carriage Class:* Semantic <fieldset> with <legend> containing inline radio buttons for Seat, Coupé, Sleeper.
5. *Phone & Email:* 2-column row where Phone utilizes an input group (.input-group) with a teal +7 prefix badge.
6. *SMS & Agreement:* Bootstrap switch (.form-switch) for SMS and mandatory checkbox for carriage terms.

Color Palette & Proportions: Enforces the assignment palette: Primary Teal (#0F6B5F), Accent Amber (#F2A93B), Dark Green (#16302B), Light Background (#F4F7F5), and Pale Green (#E6EFEC). The card has rounded corners (20px) and subtle elevation.

Responsive Adaptation & Mobile Button Order: On desktop (1280 px), action buttons align right with Clear on left and Find trains on right. On mobile (375 px), the form stacks in a single column. To position **Find trains above Clear on a phone without changing HTML source order**, the button row uses display: flex with flex-direction: column-reverse in @media (max-width: 575.98px). The Clear button (<button type="reset">) restores the form state, and keyboard navigation is verified.

[SCREENSHOT PLACEHOLDER — TASK 1 / 375 PX] **Mobile Viewport (375px)** **Find trains stacked above Clear**	**[SCREENSHOT PLACEHOLDER — TASK 1 / 1280 PX]** **Desktop Viewport (1280px)** **Full Form Layout & Proportions**

Figure 1: Side-by-side comparison of the train-ticket booking interface at 375 px (phone) and 1280 px (laptop/desktop). Left: Mobile layout showing single-column stacking and Find trains above Clear. Right: Desktop layout with multi-column alignment.

Task 2 — Registration for AITU Hack 2026 (HTML-Only Validation)

Native HTML5 Constraint Validation: All 13 validation rules are enforced purely using native HTML attributes:
• Semantic types: email, number, date, time, url, password.
• Precise step and boundary attributes: Age (min=16, max=30, step=1), Team size (min=1, max=4, step=1), Arrival day (min=2026-11-20, max=2026-11-22), Check-in time (min=09:00, max=18:00, step=1800 for 30 min intervals), Entry fee (min=0, max=5000, step=500).
• Length bounds: Full name (minlength=2, maxlength=40), Password (minlength=8, maxlength=20), Motivation (minlength=50, maxlength=300).
• Grouping: Track dropdown with placeholder option value="", Experience level radio buttons sharing name="experience" with required.

Interaction-Aware Feedback via CSS :user-invalid: To prevent error indicators on initial page load, custom CSS leverages :user-invalid and :user-valid. Sibling .invalid-feedback elements become display: block only when their associated input is :user-invalid. Parent containers (.input-group:has(:user-invalid), fieldset:has(:user-invalid), .form-check:has(:user-invalid)) display feedback reliably.

Field	Constraint Rule	Must Accept	Must Reject	Status
Full name *	minlength=2, max=40	Anna-Maria Lee	A (< 2 chars)	Verified Pass
University email *	type=email, required	a.fedenko@astanait.edu.kz	a.fedenko, a.fedenko@	Verified Pass
Age *	min=16, max=30, step=1	18	15, 31	Verified Pass
Team size *	min=1, max=4, step=1	4	5, 2.5	Verified Pass
Track *	select required, value=""	Web	“Choose…”	Verified Pass
Arrival day *	2026-11-20 to 2026-11-22	21.11.2026	25.11.2026	Verified Pass
Check-in time *	09:00–18:00, step=1800	10:30	10:15, 19:00	Verified Pass
Entry fee *	0–5000 ■, step=500	1500	1250, 6000	Verified Pass
Experience level *	3 radios, same name, req	Beginner	Nothing chosen	Verified Pass
GitHub profile	type=url (optional)	(empty), https://github...	github.com/yerassyl-dev	Verified Pass
Password *	minlength=8, max=20	Astana2026	short (< 8 chars)	Verified Pass
Motivation *	minlength=50, max=300	120 characters	49 characters	Verified Pass
Rules agreement *	type=checkbox, required	Ticked checkbox	Not ticked	Verified Pass

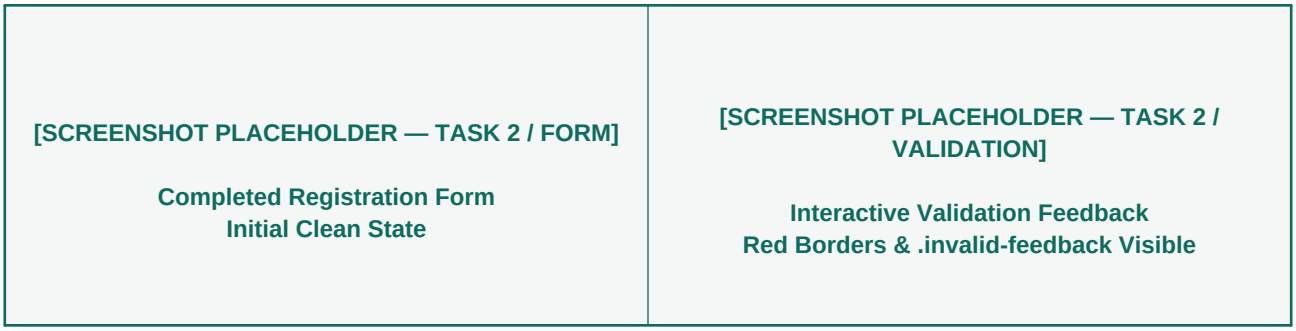

Figure 2: Task 2 implementation evidence. Left: Completed registration form displaying all 13 controls. Right: Live validation feedback showing red borders and .invalid-feedback messages displayed after user interaction.

Task 3 — Fixing the Defective Café Reservation Form

Audit of Original Implementation: The original café reservation form suffered from severe usability, accessibility, and structural defects. An audit identified 14 distinct issues in the source code, including missing labels, generic input types, omitted name attributes leading to lost submissions, duplicate IDs, uncoupled radio inputs, and an inert button.

Original Code Snippet	Problem & User Impact	Correction Implemented	Alternative Fix & Rationale
`<input type="text" placeholder="Your name">`	No label; placeholder used as label replacement. Disappears when typing, causing loss of context. Missing name="name" prevents submission.	Added <label for="name">Your name</label> and id="name" name="name" required.	*Alternative Fix:* Implicit label wrapping (<label>Your name <input...></label>). *Rationale:* Explicit for/id binding chosen for styling flexibility with Bootstrap grid and reliable screen reader support.
`<input type="text" placeholder="Email">`	Generic type="text" without label or name. Mobile devices show text keyboard instead of email keypad. Browser cannot validate email format.	Added label, changed to type="email", added id="email" name="email" required.	—
`<label>Phone</label><input id="phone">`	Label lacks for attribute; clicking label does not focus input. Uses type="text" instead of type="tel". Missing name attribute.	Added for="phone" to label, changed to type="tel", added name="phone" required.	—
`<label for="guests"><input id="people">`	Mismatched binding: for="guests" points to non-existent id. Screen readers cannot link label. Allows non-numeric text.	Reconciled ID to id="guests", changed to type="number" min="1" max="20", added name="guests" required.	—
`<input id="date" placeholder="dd/mm/yyyy">`	Text input with placeholder instead of native calendar control. Prone to formatting errors. Missing name="date".	Changed to type="date", added name="date" and required.	—
`<input type="radio" name="inside" id="date">`	Duplicate id="date" violates W3C uniqueness rule. Raw unlinked text 'Inside'.	Assigned unique id="seatingInside", wrapped text in <label for="seatingInside">.	—
`name="inside" vs name="terrace"`	Different name attributes on radios prevent grouping, allowing user to check both options simultaneously.	Unified name attribute to name="seating" with values 'inside' and 'terrace'.	—
`<p>Seating:</p>`	Unsemantic <p> paragraph tag fails to group radio choices for assistive technologies.	Replaced with semantic <fieldset><legend>Seating</legend>...</fieldset>.	—
`<select><option>Lunch...</select>`	Select has no label, id, or name. Meal choice is never sent in submission.	Added label for meal, id="meal", name="meal", and explicit option values.	—
`<textarea placeholder="Comments">`	Missing label, id, and name. Comments discarded on submit.	Added <label for="comments">, id="comments", name="comments".	—
`<input type="checkbox"> I agree...`	Checkbox has unlinked text, no id, no name. Missing required allows reserving without agreement.	Added id="terms", name="terms", required, and explicit <label for="terms">.	—
`<div class="btn btn-primary">Reserve</div>`	is not keyboard focusable, cannot be activated via Enter/Space, and does not submit form data.	Replaced with native <button type="submit" class="btn btn-reserve w-100">Reserve</button>.	*Alternative Fix:* Add tabindex="0", role="button", and JS submit handler. *Rationale:* Native button chosen for built-in keyboard accessibility, accessibility tree support, and zero JavaScript compliance.
`<form>`	No action or method attributes; defaults to GET appending data to browser URL.	Added action="#" method="post".	—

`Overall Layout`	Unstyled inline elements lacking grid wrappers or spacing; breaks on mobile.	Wrapped in Bootstrap .container, .row, .col-md-6, .mb-3 inside a centered card.	—

W3C Validation Verification: The rebuilt task3.html file was submitted directly to the official W3C Markup Validation Service (https://validator.w3.org/nu/). The file passed with **ZERO errors and ZERO warnings**.

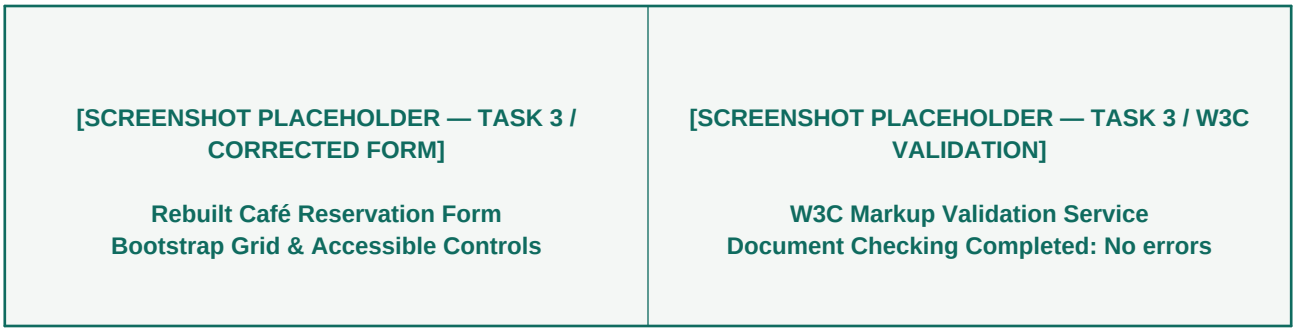

Figure 3: Task 3 implementation evidence. Left: Corrected café reservation form rebuilt with Bootstrap 5.3. Right: Official W3C Markup Validation Service result confirming zero markup errors.

Conclusion

Completing Assignment 4 provided deep practical insight into the design, validation, and debugging of modern web forms. Through Task 1, I mastered the integration of Bootstrap 5.3 form components with custom CSS overrides. I learned how components like button checks (.btn-check), floating labels (.form-floating), input groups, switches, and custom selects can be restyled to match an exact brand design while preserving responsive agility. Implementing the mobile button reordering via flex-direction: column-reverse reinforced the power of mobile-first CSS flexbox over fragile DOM manipulation.

In Task 2, I discovered the immense expressive power of native HTML5 constraint validation. By utilizing specialized input types, precise numerical steps, temporal intervals, and character length boundaries, I verified that sophisticated data constraints can be enforced completely without JavaScript. Coupling these native constraints with modern CSS pseudo-classes (:user-invalid and :user-valid) solved a fundamental UX challenge: preventing premature error alerts when a user first lands on a page while providing instant, accessible feedback the moment they modify a field.

Finally, Task 3 was an invaluable lesson in defensive markup architecture and accessibility. Auditing the defective café form highlighted how seemingly subtle errors—such as missing name attributes, broken for/id relationships, duplicate IDs, and non-semantic buttons—completely break form submission, keyboard navigation, and screen reader compatibility. Rebuilding the form cleanly to pass the W3C validator with zero errors proved that semantic HTML is not just an academic formality, but the foundational bedrock of a reliable, universally accessible web.